TRABAJO PRACTICO ESPECIAL

Sistemas Embebidos

Descripción breve

En este informe se hará un análisis detallado del trabajo practico especial, el mismo consiste en el desarrollo de un reproductor de archivos WAV.

Jorge Tobias Burdisso
tobiasburdisso@gmail.com

Introducción

En el presente trabajo diseñé e implementé un reproductor de audio WAV sobre el microcontrolador STM32F103C8, utilizando FreeRTOS (CMSIS-V2) y STM32CubeIDE. El sistema lee archivos WAV (mono/8-bits/8kHz) desde una tarjeta SD por SPI, aplica un filtro digital en cascada de dos bandas, reproduce el audio mediante una salida PWM a 100 kHz y, simultáneamente, realiza un análisis espectral (FFT de 64 puntos) para mostrar una representación tipo ecualizador en una pantalla OLED SSD1306. También implementé una interfaz de usuario controlada por un joystick analógico para seleccionar, reproducir y pausar canciones.

El desafío principal consistió en garantizar la reproducción continua y sin cortes del audio mientras se realizan operaciones de cálculo (filtrado, FFT) y de entrada/salida (lectura SD, actualización de la pantalla y generación PWM). Para ello utilice una arquitectura basada en tareas RTOS, sincronizadas mediante colas, semáforos y mutex: una tarea de lectura SD que alimenta buffers ping-pong y envía bloques de datos a la tarea FFT; una tarea FFT que procesa estos bloques y prepara el buffer de la pantalla; y una tarea de display que se encarga exclusivamente de actualizar la OLED. La conversión ADC del joystick y la selección de archivos se integran en la tarea UI, por otra parte, la reproducción temporal se gestiona por una ISR que interrumpe a una frecuencia de 8kHz que alimenta el PWM.

Este informe muestra el diseño y la implementación del sistema. Presento resultados experimentales de funcionamiento (uso de CPU, mediciones con analizador lógico, uso de memoria) y limitaciones del sistema (restricción del tiempo de ejecución de las tareas debido a la frecuencia de muestreo de 8kHz y limitaciones de memoria). Finalmente, detallo conclusiones y posibles mejoras que puedan mejorar el desempeño del sistema.

Descripción general y solución propuesta.

A continuación, en la *figura 1,* detallo un esquema sencillo de la solución propuesta. Este sirve de referencia para ver la relación entre las tareas, de que se encarga cada una, como el microcontrolador divide el trabajo y como es la conexión con los periféricos.

El sistema está formado por cuatro tareas y una ISR. La tarea de mayor prioridad es **TASK_FILTRO_SD**, responsable de la lectura SD por SPI, el preprocesado de las muestras (normalización y filtrado IIR de dos bandas), el llenado del buffer ping-pong (512 bytes cada uno) con valores de ciclo de trabajo, el envío de bloques de 64 muestras a la cola (no bloqueante) de la FFT y avisar a la tarea display a través del semáforo (**semUPDATE**) que actualice a través de I2C la pantalla. Para garantizar una reproducción sin cortes o alteraciones y el suministro fiable de datos a la FFT, se le asigna la máxima prioridad.

La tarea que le sigue en prioridad es la tarea **TASK_FFT,** esta recibe a través de una cola un bloque de 64 muestras. A estas se les calcula la transformada y luego se utiliza un algoritmo para graficar las barras y lograr el efecto de visualización. Esto se carga luego en la RAM para que la tarea display lo único que tenga que hacer es actualizar la pantalla y no desperdicie

tiempo generando el grafico.

La próxima en prioridad es la tarea **TASK_Display**, y como ya mencione, lo único que realiza es actualizar la pantalla (encargada de la comunicación I2C), para lograr esto utilizo el semáforo **semUPDATE.** Si bien durante la reproducción de los archivos el único que se encarga de liberar este semáforo es **TASK_FILTRO_SD** también es utilizado por otras funciones diseñadas por mi para facilitar y simplificar la programación.

La ultima en prioridad es la tarea **TASK_UI**, la cual toma los valores del ADC. Se encarga de la animación de una flecha para la selección de la canción como también, ir cambiando un índice para que cuando elija la canción se lo mande a **TASK_FILTRO_SD** a través de la cola **SongIndex** y esta sepa que archivo tiene que comenzar a leer.

Por último, la interrupción, encargada de modificar el ciclo de trabajo de la PWM. Consume las muestras del buffer ping-pong que alimenta **TASK_FILTRO_SD.** Cuando esta termina de leer todas las muestras disponibles en el buffer, realiza un swap entre buffers y comienza a consumir el otro ya escrito, después libera el semáforo **semRW** para que **TASK_FILTRO_SD** comience a escribir nuevamente. De esta manera me aseguro que la ISR tenga siempre muestras disponibles y no haya sobre escrituras o lecturas.

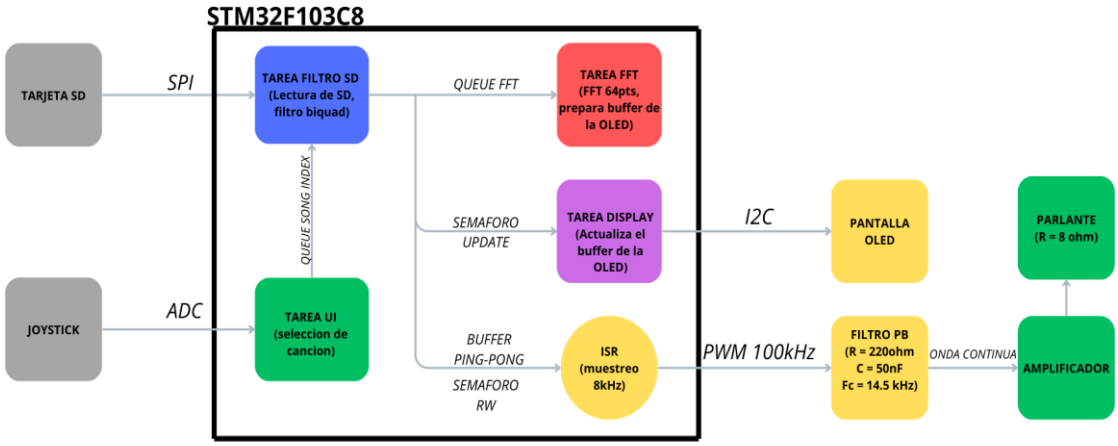


Figura 1.

Además, se puede ver las etapas luego de generar la señal PWM en donde utilizamos un filtro pasa bajo con una frecuencia de corte de:

$$\mathrm{fc} = \frac{1}{2\pi \mathrm{RC}} = \frac{1}{2\pi * 220\Omega * 50\mathrm{nF}} = 14.47\ \mathrm{kHz}$$

Elegí esta frecuencia debido a que atenúa eficazmente la portadora de la PWM (~100KHZ) y el ruido de conmutación, pero deja pasar holgadamente la banda útil del audio. De esta forma, el filtro digital elimina las componentes cercanas a la frecuencia de Nyquist ($f_{max} = {}^{fs}/_{2} = 4kHz$) y el analógico limpia la portadora y las frecuencias residuales más altas.

La señal luego es amplificada por un amplificador PAM8403. Este es un amplificador clase D de 3W de potencia. Permítela la salida de audio estéreo, pero en mi caso como el audio es mono, utilizare un solo parlante.

Detalles de la implementación

En este apartado iré detallando aspectos relevantes del programa, funciones, descripción del filtro digital utilizado y aspectos que ayuden a un mejor entendimiento del sistema.

Funciones

La primera de las funciones que voy a detallar es **ScanWavFiles()**. Su función es buscar el nombre de los archivos WAV de la tarjeta SD y guardarlo en un arreglo llamado *wavFiles[MAX_WAV_FILES][MAX_NAME_LEN]*, de esta manera los nombres los puedo mostrar en la pantalla una vez inicializadas las tareas.

Las próximas funciones son **iniciar_reproducciom_pwm()** y **detener_reproduccion_pwm()** que ya como el nombre indica sirve para comenzar y detener la reproducción de los archivos. Estas fueron de suma importancia ya que la primera asegura un correcto inicio de la reproducción, inicializando timers de la rutina de interrupción y el PWM, mostrando la vista de la pantalla de ecualización y enviando a la tarea TASK_FILTRO_SD el índice de la canción seleccionada. Por su parte la otra, detiene la rutina de interrupción y el PWM, mueve nuevamente a los punteros de los buffers ping-pong a su posición inicial, reinicia las variables de los filtros para evitar que queden datos de la canción anterior, limpia la cola de la FFT para que este vacía al momento de reproducir otra, asegura un correcto cierre del archivo WAV y finalmente vuelve a mostrar el menú de selección de canción.

La función **fast_sqrtf(float** number**)** es una función la cual sirve para estimar la raíz cuadrada utilizando el método de Newton-Raphson. Debido a que la función **sqrt()** de la librería **"math.h"** ocupaba mucho espacio opte por esta solución debido a su rapidez, bajo costo computacional y como la exactitud no era mi mayor interés debido a que sirve para calcular el alto de las barras de ecualización era una perfecta solución.

Por último la función **filtro_iir(float** x, **Biquad_t** *f**)** recibe una muestra de audio (x) y un puntero a una estructura *Biquad_t* (diseñada por mi) que contiene los coeficientes del filtro y las variables de estado (entradas y salidas anteriores). Implementa la ecuación de diferencia de un filtro con respuesta infinita al impulso en forma directa I (Direct Form I), la cual utiliza dos memorias para las entradas previas y dos para las salidas previas. Este tipo de filtro es un biquad (segundo orden), por lo que posee dos polos y dos ceros. La forma del filtro es:

$$y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-2]$$

Figura 2

En este proyecto utilice dos biquads en cascada: un pasa altos de 200 Hz y un pasa bajos de 3600 Hz, diseñados con la herramienta earlevel.com. El filtro pasa alto atenúa el contenido de muy baja frecuencia (como DC offset y zumbidos), limpiando la señal antes de procesarla. El pasa bajos reduce las componentes por encima de la banda útil (Nyquist = 4 kHz para Fs=8 kHz), previniendo aliasing y limitando el espectro antes del filtrado analógico. De complemento para esta función utilizo **filtro_reset(Biquad_t** *f**)** la cual permite reiniciar las variables internas para que no queden datos de las ejecuciones anteriores.

Memoria RAM y Flash

Para ver la memoria utilizada en nuestro programa tanto la RAM como la FLASH utilice la herramienta incluida en el STMCubeIDE la cual al entrar en una sesión de debug puedo ir al Build Analyzer, ver el uso de tanto la memoria RAM como FLASH y su porcentaje de uso.

Figura 3.

Region	Start address	End address	Size	Free	Used	Usage (%)
RAM	0x20000000	0x20004fff	20 KB	1,34 KB	18,66 KB	93.32%
FLASH	0x08000000	0x0800ffff	64 KB	920 B	63,1 KB	98.60%

Name	Run address (VMA)	Load address (LMA)	Size
RAM	0x20000000		20 KB
._user_heap_stack	0x200044a4		1,5 KB
.bss	0x20000880		15,04 KB
.data	0x20000000	0x0800f3ec	2,12 KB
FLASH	0x08000000		64 KB
.data	0x20000000	0x0800f3ec	2,12 KB
.fini_array	0x0800f3e8	0x0800f3e8	4 B
.preinit_array	0x0800f3e4	0x0800f3e4	0 B
.init_array	0x0800f3e4	0x0800f3e4	4 B
.rodata	0x0800e090	0x0800e090	4,83 KB
.text	0x08000110	0x08000110	55,88 KB
.isr_vector	0x08000000	0x08000000	268 B

Figura 4.

Como se ve, estoy utilizando un gran porcentaje de la memoria, tanto FLASH como RAM. Esto se debe a que en la FLASH tengo almacenado todo el programa y las librerías necesarias: el código, las funciones de la CMSIS, el módulo FatFs para leer la tarjeta SD, la librerias FFT y SSD1306 para el manejo de la pantalla OLED. Por otro lado, la RAM la utilizo para todo lo que ocurre en tiempo real: buffers grandes (por ejemplo, los dos buffers ping-pong), el buffer de la OLED donde se construye la imagen antes de enviarla por I2C, las matrices temporales de la FFT y las variables internas de los filtros. También cada tarea utiliza su stack para almacenar variables locales y el sistema reserva memoria para las colas, semáforos y estructuras de coordinación entre tareas e ISRs.

Se puede ver el tamaño designado del HEAP desde el .ioc el cual este es administrado por mi y lo elegí en base al espacio designado para las tareas.

HEAP STILL AVAILABLE	895 Bytes
TOTAL HEAP USED	7105 Bytes
Total amount for tasks	6312 Bytes
Total amount for queues	441 Bytes
Total amount for timers	0 Bytes
Total amount for mutexes and semaphores	352 Bytes
Total amount for events	0 Bytes

Figura 5.

En donde las tareas, semáforos y colas ocupan:

Task Name	Priority	Stack Size (Words)
TASK_FILTRO_SD	*AboveNormal6*	*650*
TASK_FFT	*AboveNormal5*	*350*
TASK_Display	*AboveNormal4*	*150*
TASK_UI	*AboveNormal3*	*150*

Semaphore Name	Initial State
semRW	*Depleted*
semUPDATE	*Depleted*
semCLOSE	*Depleted*

Queue Name	Queue Size	Item Size
Queue_SongIndex	*1*	*1*
Queue_FFT	*1*	*256*

Figura 6.

Por último, hago uso de la herramienta del STMCubeIDE, *Static Stack Analyzer* para ver en tiempo real cuanto ocupa en memoria cada tarea y de *FreeRTOS Task List* donde se ve el estado de cada tarea durante la ejecución.

Function	Depth	Max cost	Local cost
ADC1_2_IRQHandler	?	?	0
xTASK_FILTRO_SD	9	2240	1376
Reset_Handler	13	1456	0
xTASK_FFT	6	1072	848
xTASK_UI	9	484	24
xTASK_Display	6	436	16

Figura 7.

El *max cost* es una estimación que hace el programa donde calcula el espacio máximo que puede ocupar la tarea debido a llamadas a funciones en el peor de los casos.

	TASK_Display	36/36	0x20003d50	0x20003ef0...	BLOCKED	semUPDATE
	TASK_FFT	37/37	0x20003768	0x200038e...	BLOCKED	Queue_FFT
	TASK_FILTRO_SD	38/38	0x20002d98	0x200030f0...	BLOCKED	Queue_Son...
→	TASK_UI	35/35	0x20002ad0	0x20002ce...	RUNNING	

Figura 8

	TASK_Display	36/36	0x20003d50	0x20003ef0...	BLOCKED	semUPDATE
→	TASK_FFT	37/37	0x20003768	0x200038e...	RUNNING	
	TASK_FILTRO_SD	38/38	0x20002d98	0x200030f0...	BLOCKED	semRW
	TASK_UI	35/35	0x20002ad0	0x20002cc8...	READY	

Figura 9.

→	TASK_Display	36/36	0x20003d50	0x20003ef0...	RUNNING	
	TASK_FFT	37/37	0x20003768	0x200038e...	BLOCKED	Queue_FFT
	TASK_FILTRO_SD	38/38	0x20002d98	0x200030f0...	BLOCKED	semRW
	TASK_UI	35/35	0x20002ad0	0x20002cb...	READY	

Figura 10.

	TASK_Display	36/36	0x20003d50	0x20003ef0...	BLOCKED	semUPDATE
	TASK_FFT	37/37	0x20003768	0x200038e...	BLOCKED	Queue_FFT
→	TASK_FILTRO_SD	38/38	0x20002d98	0x200030f0...	RUNNING	
	TASK_UI	35/35	0x20002ad0	0x20002c4...	READY	

Figura 11

Factor de utilización

Para medir el factor de utilización de cada tarea y el IDLE utilicé task hooks y el analizador lógico, donde obtuve el tiempo activo de cada una y también la del IDLE.

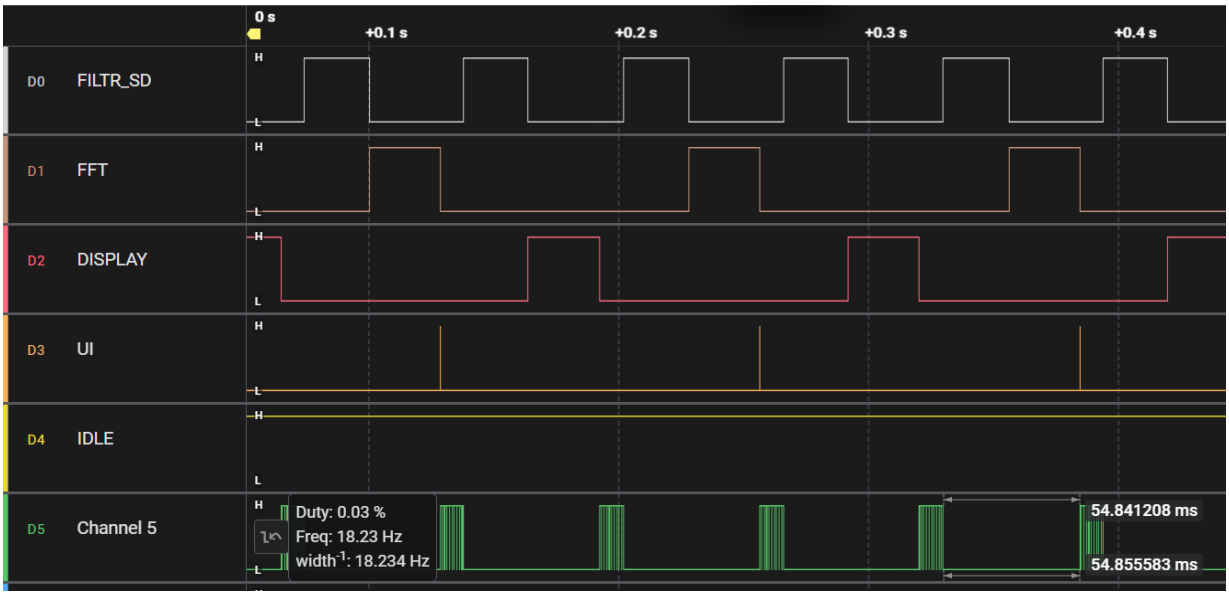


Figura 12.

Como la ejecución del IDLE es periódica puedo calcular el factor de utilización del sistema con esta, donde el tiempo en la que se encuentra en alto es el tiempo libre del sistema, por lo que calculamos.

$$U = \frac{9.24ms}{54.46ms} = 0.17 = 17\%$$

Osea que el sistema está un 17% libre.

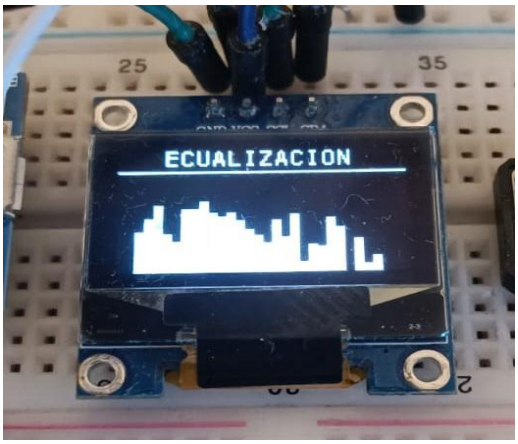
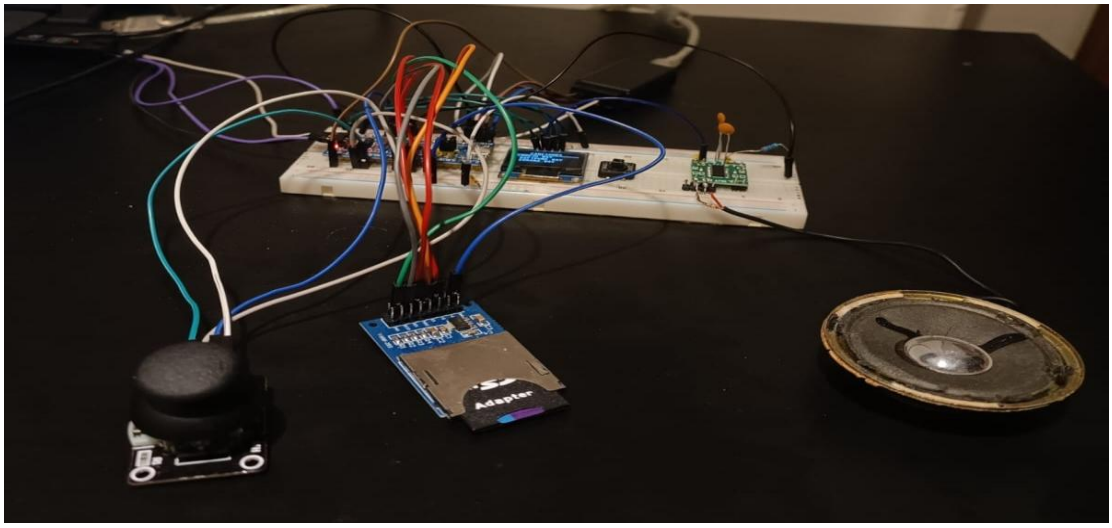
La ISR de muestreo se dispara cada 125 µs (8 kHz) y marca “buffer completo” cada 512 muestras, es decir, cada 64 ms. La rutina dentro de la ISR es breve (actualiza PWM y notifica), por lo que no impacta de forma apreciable en los márgenes calculados.

El factor de utilización del sistema es 83%. Aunque es alta, la captura muestra que todas las tareas completan su trabajo antes de su siguiente liberación: la ruta crítica es FILTRO_SD frente al período de 64 ms y tiempo de ejecución de 23ms, debido a que tiene que cumplir este plazo para mantener lleno el buffer ping-pong así el audio se escucha óptimo. FFT y DISPLAY operan a 128 ms con más de 100 ms de holgura, esto lo diseñe así haciendo uso de una variable *var* en la tarea FILTRO SD, para que en cada tiempo muerto de la tarea FILTRO SD se ejecute una de las dos. De esta manera cada una podía terminar la tarea que estaba realizando, ya que tuve el problema de que cuando se ejecutaba una, la otra comenzaba, pero debido al plazo estricto de FILTRO SD esta la interrumpía haciendo que nunca se termine de ejecutar. En ejecución real no aparecen glitches en el audio ni congelamientos: la combinación de prioridades, semáforos y buffers ping-pong asegura que FILTRO_SD siempre alimente a tiempo al ISR y que FFT/DISPLAY se ejecuten en las ventanas libres.

Conclusión

En este trabajo implemente un reproductor de audio WAV sobre un STM32F103C8, capaz de leer archivos desde una tarjeta SD mediante SPI, filtrarlos con un sistema digital en cascada (pasa alto 200 Hz y pasa bajo 3.6 kHz), generar la señal PWM para la reproducción y mostrar en tiempo real la ecualización calculada por una FFT en una pantalla OLED. Todo el sistema funciona bajo FreeRTOS, con cuatro tareas principales sincronizadas por colas y semáforos, logrando una reproducción continua y una interfaz fluida. Conseguí que el CPU trabaje con un alto factor de utilización sin afectar la estabilidad del audio, gracias a los buffers ping-pong, semáforos, colas y a la correcta asignación de prioridades entre tareas. El filtrado digital y la FFT se ejecutan en paralelo a la reproducción, aprovechando los intervalos libres de lectura.

Entre las limitaciones, se destaca que la frecuencia de muestreo es fija en 8 kHz, lo que no solo limita la calidad máxima de audio reproducible, sino que también determina la velocidad de ejecución de la tarea de FILTRO SD y acorta el tiempo que tienen las otras tareas para ejecutarse. Otra limitación importante es el uso elevado de la memoria RAM y FLASH, debido a la inclusión de librerías como FreeRTOS, FatFs, controladores de pantalla y rutinas FFT, junto con el uso de buffers grandes. Esto deja un margen reducido para agregar nuevas

funcionalidades sin optimizar previamente el código o la gestión de memoria. Intente utilizar las herramientas de optimización de código del entorno sin éxito ya que afectaban a la funcionalidad del mismo.

Como mejoras, propongo incorporar DMA para liberar carga de la CPU, aumentar la memoria para soportar mayor funcionalidad, y utilizar un reloj de mayor velocidad que permita trabajar con frecuencias de muestreo superiores, ampliando la respuesta en frecuencia y la calidad de reproducción.